LLM-ASSISTED MULTI-AGENT STOCK DECISION SUPPORT SYSTEM FOR DAILY-FREQUENCY SIMULATED TRADING

FINAL REPORT FOR REPRESENTATION AND REASONING: FOUNDATIONS OF LARGE MODELS

Mo Di (2021030019)
Applied Track Course Project
GitHub: modi20230416/stock_agent

June 17, 2026

ABSTRACT

This applied-track project builds a reproducible multi-agent stock decision support prototype for daily-frequency simulated trading. The system addresses three tasks: single-stock analysis, candidate screening, and small-portfolio rebalancing under risk constraints. It combines specialized agents for market information, news sentiment, fundamental analysis, risk management, and final decision synthesis, with an optional OpenRouter-based LLM reviewer that reads structured intermediate evidence rather than raw prompts alone. To make the project reproducible without paid data services, I constructed a deterministic offline dataset covering 12 large-cap tickers, 60 trading days, sample news events, fundamentals, and portfolio constraints. The final benchmark suite contains 15 fixed cases: 10 single-stock cases, 2 screening cases, and 3 rebalancing cases. The system passes all 15 cases, validates the dataset schema, maintains explicit risk warnings, satisfies portfolio constraints, and reports baseline comparisons against a technical-rule baseline, a no-risk ensemble baseline, and a direct LLM/single-agent baseline. It also produces equal-weight and decision-weighted backtests plus portfolio stress tests. The final deliverable includes runnable code, tests, Markdown reports, a static HTML dashboard, and usage documentation. The project demonstrates that a small but coherent multi-agent workflow can improve transparency, reproducibility, and risk awareness for simulated financial decision support.

1 Problem Definition

This project addresses the problem of building an interpretable decision-support prototype for daily-frequency stock analysis and simulated portfolio adjustment. The target users are students or beginning investment researchers who want to inspect how market, news, fundamental, and risk signals can be combined into a structured recommendation. The system is not designed for real-money trading, broker integration, or fully autonomous execution.

The final system operates on a small offline stock universe. Its inputs are daily OHLCV price bars, short news/event records, basic fundamental indicators, and portfolio constraints. Its outputs are:

- a single-stock action, score, confidence, rationale, and risk warnings;
- a ranked candidate list for a small stock pool;
- target portfolio weights, trade deltas, and constraint warnings for rebalancing;
- structured intermediate outputs from each agent.

The problem is not trivially solved by directly asking an LLM for a stock opinion. A direct prompt can produce fluent text but often hides the evidence used, mixes time horizons, ignores portfolio constraints, and is difficult to reproduce. This project therefore separates the workflow into specialized modules and asks the optional LLM component to

review structured evidence rather than invent the entire decision. The final scope is a deterministic applied prototype that can be run locally, tested automatically, and inspected through generated reports.

2 Related Work and Project Positioning

Recent work on LLM financial agents motivates the use of structured roles and explicit evidence. FinMem proposes a trading agent with profiling, layered memory, and decision-making modules, emphasizing how historical and hierarchical financial information can support interpretable decisions [Yu et al., 2023]. TradingAgents is the closest architectural reference: it uses specialized LLM-powered roles such as fundamental analysts, sentiment analysts, technical analysts, traders, and risk managers [Xiao et al., 2024]. This project adopts the core idea of role decomposition but scales it down into a course-appropriate prototype with deterministic offline data, simpler scoring rules, and explicit tests.

FinGPT highlights the data-centric challenge of financial LLM systems: useful financial models require pipelines for gathering, cleaning, and structuring financial data [Yang et al., 2023]. This influenced the decision to keep loaders separate from agents and to preserve an offline processed dataset. FinRL provides a reference point for quantitative-finance tooling, particularly its focus on reproducibility, modular pipelines, and backtest metrics such as cumulative return, Sharpe ratio, and maximum drawdown [Liu et al., 2021]. The present project does not train a reinforcement-learning policy, but it borrows the idea that trading-related systems should report risk-adjusted metrics rather than only qualitative recommendations. Finally, ReAct-style reasoning-and-acting work [Yao et al., 2022] motivated the broader principle that useful agent systems should expose intermediate actions and observations; in this project that principle appears as structured agent results and auditable JSON reports.

The main contribution of this project is engineering integration rather than a new financial model. Compared with prior large frameworks, it is smaller, fully reproducible, and focused on a fixed course benchmark. Compared with a single LLM call, it is more inspectable and constraint-aware.

3 Methodology and Implementation

3.1 Use Case and Design Goals

The concrete use case is a local decision-support tool for simulated daily stock analysis. A user should be able to run a command, select a task, and inspect why the system produced a recommendation. The design goals are:

1. **Reproducibility:** the benchmark must run without external API keys or network access.
2. **Interpretability:** every final recommendation must include intermediate evidence.
3. **Risk awareness:** portfolio outputs must respect max-position, max-trade, minimum-cash, and volatility constraints.
4. **LLM optionality:** OpenRouter review should be usable when a key is available but should not be required for tests.
5. **Small-system completeness:** the delivered artifact should include data, code, tests, baseline comparisons, reports, and usage instructions.

3.2 System Architecture and Workflow

The central class is `StockDecisionSystem` in `src/stock_agent/pipeline.py`. It loads data from CSV/JSON files, orchestrates specialized agents, and exposes five main methods: `analyze_single`, `screen_candidates`, `rebalance`, `benchmark`, and `benchmark_cases`.

Layer	Role in the implementation
Data layer	`data_loader.py` reads prices, news, fundamentals, portfolio constraints, and benchmark cases.
Agent layer	Market, news, fundamental, risk, and decision agents produce structured `AgentResult` and `Decision` objects.
LLM layer	`LLMDecisionAdvisor` optionally calls OpenRouter and parses JSON review output.
Evaluation layer	Baselines, final benchmark cases, data validation, equal-weight and decision-weighted backtests, and stress tests summarize behavior.
Report layer	`evaluator.py` writes JSON, Markdown, and a static HTML dashboard.

The workflow for single-stock analysis is: load historical data up to the chosen date; compute market, news, and fundamental scores; combine them through the decision agent; pass the tentative action through risk management; optionally ask the LLM reviewer for a structured adjustment; then return the final action with evidence and warnings. Screening runs this workflow over a candidate pool and ranks by score. Rebalancing converts decisions into target weights and applies portfolio constraints.

3.3 Key Technical Design Choices

LLM and model choice. The optional LLM integration uses OpenRouter through `src/stock_agent/llm.py`. The default free-model string is `openai/gpt-oss-20b:free`, and users can override it with `OPENROUTER_MODEL`. Three modes are supported: `off`, `auto`, and `required`. In `off` mode, no LLM call is made even if an API key is present. In `auto` mode, the system falls back to deterministic rules if no key is available. In `required` mode, a failed LLM call raises an error, which is useful for verifying a real API demonstration.

Agent orchestration and structured output. Each agent returns a typed object containing score, confidence, summary, evidence, warnings, and metrics. This makes the system easier to test and inspect than a free-form LLM answer. The decision agent uses a weighted combination of market, news, and fundamental signals, then applies a conflict penalty when strong positive and strong negative signals coexist.

Data and tools. The final offline dataset is generated by `scripts/generate_final_data.py`. It contains 12 tickers, 60 trading days, news events with event types, fundamental records, and benchmark cases. This keeps the project reproducible while still matching the proposal's goal of a 10–20 stock evaluation pool.

Interface and deployment. The primary interface is a CLI:

Command option	Purpose
`--taskvalidate`	Check dataset schema, coverage, and portfolio weights.
`--tasksingle`	Analyze one ticker.
`--taskscreen`	Rank a candidate pool.
`--taskrebalance`	Generate portfolio target weights.
`--taskbenchmark`	Compare the multi-agent system with baselines.
`--taskfinal`	Run the 15-case final benchmark suite.
`--taskall`	Generate all JSON, Markdown, and HTML outputs.

The project also writes `reports/final_dashboard.html`, a static dashboard that can be opened directly in a browser without installing Streamlit or a web server.

Reliability and failure handling. The implementation avoids hard dependency on external services. Missing API keys fall back to deterministic logic, LLM responses must parse as JSON, and tests verify that `--llmoff` prevents hidden LLM baseline calls. Portfolio outputs are checked for max-position, max-trade, minimum-cash, and sum-to-one constraints.

3.4 Implementation Scope and Tradeoffs

The final implementation includes the core applied system, benchmark data, tests, reports, and dashboard. The main simplification is that the processed dataset is deterministic rather than downloaded from live financial APIs. This is

Table 1: Final benchmark and system validation results.

Evaluation item	Result
Unit tests	7/7 passed
Final benchmark cases	15/15 passed
Final criteria	7/7 passed
Dataset validation	PASS, no warnings
Single-stock cases	10/10 passed
Screening cases	2/2 passed
Rebalancing cases	3/3 passed
Stock universe size	12 tickers
Multi-agent risk warnings	16
Technical baseline action diversity	2
No-risk baseline action diversity	3
Direct LLM baseline used LLM count in offline mode	0
Worst stress scenario	tech_ai_drawdown, -7.46%

a deliberate tradeoff: for a course final project, reproducibility and inspectability are more important than real-time market coverage. The decision-weighted backtest is still a simplified simulator, but it is sufficient to show how agent scores can be converted into a repeatable portfolio evaluation loop. Future versions could replace the offline files with cached Alpha Vantage, Finnhub, Kaggle, or SEC EDGAR adapters while preserving the same agent and benchmark interfaces.

4 Results and Deliverable

4.1 System Demonstration and Walkthrough

The delivered system can be used from the command line. Representative scenarios are:

1. **Single-stock analysis.** The user runs `pythonscripts/run_demo.py--tasksingle--tickerNVDA--llmoff`. The system analyzes NVDA's price trend, news sentiment, fundamentals, and current weight, then returns a BUY/HOLD/SELL action with evidence and warnings.
2. **Candidate screening.** The user runs `pythonscripts/run_demo.py--taskscreen--tickersAAPL,MSFT, NVDA,TSLA,AMD,WMT--llmoff`. The system analyzes each ticker and outputs a ranked list by score.
3. **Portfolio rebalancing.** The user runs `pythonscripts/run_demo.py--taskrebalance--llmoff`. The system converts stock decisions into target weights, then enforces cash, position, trade-size, and volatility constraints.
4. **Final benchmark and dashboard.** The user runs `pythonscripts/run_demo.py--taskall--llmoff`. The system writes `reports/final_benchmark_results.md`, `reports/final_benchmark_results.json`, and `reports/final_dashboard.html`.

4.2 Evaluation and Testing

The final benchmark suite contains 15 cases, all of which pass. Table 1 summarizes the main evaluation results.

The benchmark includes both an equal-weight auxiliary backtest and a decision-weighted strategy backtest, shown in Table 2. These numbers should not be interpreted as investment performance claims because the dataset is deterministic and small. They demonstrate that the reporting pipeline includes standard finance-style summary metrics and can evaluate recommendations as portfolio weights.

Table 3 gives a compact view of representative benchmark outputs. The full report is available in `reports/final_benchmark_results.md`.

Table 3: Final benchmark case summary.

Case	Task	Result	Key output
positive_ai_infrastructure	single	PASS	NVDA BUY, score 1.305

Case	Task	Result	Key output
negative_auto_demand	single	PASS	TSLA SELL, score -0.7542
defensive_retail	single	PASS	WMT BUY, score 0.82
regulatory_healthcare	single	PASS	UNH HOLD, score -0.3792
chip_margin_conflict	single	PASS	AMD HOLD, score -0.2667
mega_cap_pool	screen	PASS	top ticker NVDA
cross_sector_pool	screen	PASS	top ticker NVDA
balanced_existing_portfolio	rebalance	PASS	cash 8.00%, 1 warning
overweight_high_risk	rebalance	PASS	cash 10.00%, 7 warnings
cash_defensive_rotation	rebalance	PASS	cash 19.80%, 6 warnings

After final packaging, I also ran a live OpenRouter smoke test on June 17, 2026 with the default free model `openai/gpt-oss-20b:free` and `scripts/run\_demo.py--tasksingle--tickerAAPL--llmrequired`. The LLM review path completed successfully with `llm\_review.used=true`; the default-model AAPL output was `HOLD` with score `0.5033` and confidence `0.6729`. This validates the real API integration while the official benchmark remains deterministic and reproducible in `--llmoff` mode.

4.3 Failure Analysis and Robustness

The most important robustness decision is conservative fallback. If no OpenRouter key is set, the LLM reviewer is skipped and the deterministic multi-agent output is still valid. If `--llmoff` is selected, the direct LLM baseline is also disabled, preventing accidental network use during reproducibility checks.

The final system still has known limitations. First, the offline dataset is small and synthetic-style, so it cannot measure real market generalization. Second, the scoring rules are intentionally transparent but simple. Third, the decision-weighted backtest uses a simplified weekly rebalancing loop and does not model transaction costs, slippage, taxes, or market impact. Fourth, the benchmark checks structural success, risk constraints, and case coverage rather than profitability. These limitations are appropriate for a course prototype but should be addressed before any real financial use.

4.4 Reproducibility and Usage Instructions

The project repository is `https://github.com/modi20230416/stock_agent`. To reproduce the final results on Windows PowerShell:

1. Clone the repository and enter the project directory.
2. Create and install the local environment:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e .
```

3. Run tests:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests
```

4. Validate the dataset:

```
.\.venv\Scripts\python.exe scripts\run_demo.py --task validate --llm off
```

5. Run the complete demo:

```
.\.venv\Scripts\python.exe scripts\run_demo.py --task all --llm off
```

The main generated outputs are:

- `reports/dataset_validation.md`
- `reports/benchmark_results.md`
- `reports/final_benchmark_results.md`
- `reports/final_dashboard.html`
- JSON detail files under `reports/`, which are generated locally and ignored by Git.

Table 2: Backtest metrics over the processed offline dataset.

Metric	Equal-weight	Decision-weighted
Observations	59	59
Cumulative return	4.77%	8.65%
Maximum drawdown	-1.37%	-1.18%
Sharpe ratio	2.44	4.82
Rebalances	N/A	12
Average turnover	N/A	14.25%

To verify real LLM integration, set `OPENROUTER_API_KEY` and run:

```
.\.venv\Scripts\python.exe scripts\run_demo.py --task final --llm required
```

API keys should never be committed to the repository.

5 Team Collaboration and Workload Distribution

This report version is prepared for the implemented repository by Mo Di (2021030019). The concrete work completed in the repository includes problem scoping, project setup, data schema design, deterministic final dataset generation, multi-agent implementation, optional OpenRouter integration, benchmark design, baseline and backtest implementation, dynamic strategy evaluation, stress testing, dataset validation, Markdown/HTML report generation, GitHub packaging, and final report preparation. The repository is also organized for group collaboration through GitHub branches, pull requests, and reproducible setup instructions, so other members can inspect, run, and extend the project without changing the core environment assumptions.

6 Conclusion

This project built a runnable applied prototype for multi-agent stock decision support under a daily-frequency simulated trading setting. The final system separates market, news, fundamental, risk, and decision logic; optionally reviews structured evidence with an LLM; and produces reproducible benchmark reports. The system passes 15/15 final benchmark cases, 7/7 unit tests, and 7/7 final criteria, and it includes code, data, documentation, Markdown reports, and a static dashboard.

The main takeaway is that even a small, deterministic multi-agent workflow can make financial-decision prototypes more auditable than a direct free-form LLM answer. The final iteration now goes beyond the original proposal by adding schema validation, decision-weighted backtesting, stress testing, and a live OpenRouter smoke test. Future work should connect the same interfaces to real cached data sources, expand the benchmark to more market regimes, and make the strategy backtest more execution-realistic.

References

Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Denghui Zhang, Rong Liu, Jordan W. Suchow, and Khaldoun Khashanah. Finmem: A performance-enhanced llm trading agent with layered memory and character design, 2023. URL https://arxiv.org/abs/2311.13743.

Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework, 2024. URL https://arxiv.org/abs/2412.20138.

Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. Fingpt: Open-source financial large language models, 2023. URL https://arxiv.org/abs/2306.06031.

Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. Finrl: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance*, 2021. doi: 10.1145/3490354.3494366. URL https://doi.org/10.1145/3490354.3494366.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022. URL https://arxiv.org/abs/2210.03629.

A AI Use During Project Development and Reflection

A.1 AI Use Records

AI coding assistance was used throughout the project. The main tool was Codex/ChatGPT-style coding assistance inside the local project workspace. AI was used for project planning, repository inspection, code generation, debugging, test design, report drafting, and Git/GitHub workflow support. The generated code and documentation were checked by running unit tests, inspecting output files, and validating final benchmark reports.

- **Example 1: moving from minimal version to final benchmark.**
  - **Project stage and problem:** After the minimal prototype, the system still needed a 10–20 stock benchmark, stronger agents, baselines, and final reports.
  - **AI tool and usage:** I asked the coding assistant to inspect the repository, compare it against the proposal, and implement the final-version roadmap.
  - **AI response and revision:** The assistant generated a deterministic processed dataset, benchmark cases, backtest code, baseline code, stress-test code, dataset validation, and tests. I verified the outputs by running `unittest` and the final CLI demo.
  - **Final decision or lesson learned:** AI was useful for turning a broad requirement into concrete files and tests, but the benchmark criteria required human judgment to avoid overfitting to a wording-based expected-focus field.
- **Example 2: LLM integration and offline reproducibility.**
  - **Project stage and problem:** The project needed to call a real LLM when possible while remaining reproducible without API keys.
  - **AI tool and usage:** I used the assistant to design `auto`, `required`, and `off` modes for OpenRouter integration.
  - **AI response and revision:** The assistant implemented JSON parsing, fallback behavior, and tests ensuring that `--llmoff` disables both the review agent and direct LLM baseline.
  - **Final decision or lesson learned:** LLM calls are useful for review and explanation, but the system should never depend on a hidden network call to pass tests or reproduce a report.

A.2 Unresolved Problems Despite AI Use

The largest unresolved issue is real-data generalization. AI assistance helped design a clean adapter-friendly architecture, but it cannot replace the need to obtain, clean, cache, and validate real historical price, news, and SEC fundamental data. The final project therefore uses deterministic offline data and clearly labels the system as a course prototype rather than an investment product. Human judgment was also needed to keep the evaluation honest: passing a structural benchmark does not imply profitable real-world trading.